TECHNICAL SPECIFICATION & ARCHITECTURAL OVERVIEW

# MARK: Modular Augmented Reasoning & Knowledge Assistant

A Privacy-Centric, Local-First Cognitive Architecture with Sandboxed Tool Execution, Native MCP Bridge, and Resilient Multi-Tiered Device Federation

---

| | |
|---|---|
| **Document Identifier:** | MARK-SPEC-2026-PUB-V1.1 |
| **Distribution Classification:** | PUBLIC RELEASE SPECIFICATION |
| **Release Target:** | MARK Public Edition (Branch: main) |
| **Publication Date:** | October 2026 |
| **License:** | GNU Affero General Public License v3.0 (AGPL-3.0) |
| **Target Audience:** | Systems Architects, Applied AI Engineers, Security Evaluators |
| **Authoritative Repository:** | https://github.com/thu12dta3/MARK-PUBLIC |
| **System Core Implementation:** | Python 3.10-3.14, Rust 2021 (PyO3 cdylib & native mark_link daemon), Kotlin / Jetpack Compose |


## ABSTRACT

This specification documents the architecture, cryptographic foundations, request execution pipelines, and security mechanisms implemented in the public edition of the MARK cognitive assistant. MARK delivers an autonomous, local-first artificial intelligence platform operating primarily across Linux desktop workstations with authenticated companion nodes on Android and Wear OS devices. The system incorporates local vision-language model inference via Ollama (Qwen-VL), hardware-accelerated speech recognition and synthesis, lexical and hybrid retrieval-augmented generation (RAG) with strict prompt-injection delimiters, native Model Context Protocol (MCP) tool integration in Rust with real capability isolation and memory limits, process isolation via Linux Bubblewrap namespaces, and an authenticated AES-256-GCM cryptographic vault. Furthermore, MARK Link establishes a resilient, peer-to-peer and relay-assisted transport layer featuring mutual P-256 ECDH/ECDSA authentication, transcript signing, session failover, and request idempotency. All architectural assertions, operational parameters, and component boundaries documented herein reflect verified, audited source code in the public repository.

# TABLE OF CONTENTS


| | |
|---|---|
| **1. Executive Summary** | Comprehensive mission overview and technical scope |
| **2. Scope and Design Philosophy** | Local-first computation, modularity, and boundary defense |
| **3. System Architecture** | Structural subsystem breakdown, dependencies, and interfaces |
| **4. Request Lifecycle** | End-to-end tracing of user prompt and sensory pipelines |
| **5. Model Integration and Inference** | Ollama integration, Qwen-VL reasoning, and cloud fallbacks |
| **6. Audio and Conversational Interfaces** | VAD energy detection, Whisper STT, and Piper ONNX TTS |
| **7. Vision and Multimodal Processing** | RTSP/RTSPS frame capture, buffer validation, and analysis |
| **8. Persistent Memory and Knowledge Retrieval** | Scoped memory, SQLite agenda, and untrusted RAG |
| **9. Tools, Connectors, and Authorization** | Deterministic chemistry/math registry and permission gates |
| **10. Native MCP Bridge & Capability Isolation** | Rust MCP subsystem, memory quotas, and path normalization |
| **11. Resilient Network Architecture (MARK Link)** | MLNK protocol, mutual auth, failover, and blind relay |
| **12. Desktop, Mobile, and Wearable Architecture** | PyQt6 desktop, JNI Android client, and Wear OS companion |
| **13. State Management and Resource Lifecycle** | Model unloads, process timeouts, and buffer purges |
| **14. Security and Privacy Architecture** | Bubblewrap sandboxing, AES-256-GCM vault, log redactor |
| **15. Testing and Evaluation** | Full test suite, 72 Rust tests, Bandit static scans |
| **16. Engineering Use Cases** | Workstation assistance, multimodal inspection, voice calls |
| **17. Deployment and Operations** | Installation procedures, configuration, and systemd units |
| **18. Limitations and Failure Modes** | VRAM limits, device matrix, and full-duplex boundaries |
| **19. Extensibility and Research Directions** | Plugin architecture, verified tool extensions, future work |
| **20. Conclusion and References** | Summary of release status, integrity, and operational model |

## 1. Executive Summary

The MARK (Modular Augmented Reasoning & Knowledge Assistant) project provides a robust, extensible personal AI workstation assistant designed specifically for environments where privacy, local autonomy, and verifiable execution security are paramount. Modern cloud-dependent assistant architectures introduce severe systemic risks: persistent exposure of ambient audio and visual streams to third-party telemetry, unpredictable latency variations, vendor lock-in, and susceptibility to indirect prompt injection through unpartitioned context retrieval.

MARK addresses these engineering challenges through a strictly compartmentalized architecture: neural language inference executes locally by default using instruction-tuned vision-language models (e.g., Qwen-VL); external tools are grounded in deterministic, mathematically verifiable algorithms and sandboxed native Rust Model Context Protocol (MCP) bridges; retrieved context is quarantined in isolated prompt segments; experimental workloads are jailed inside Linux Bubblewrap unshared namespaces; and inter-device connectivity is shielded by MARK Link—a proprietary, lightweight zero-trust protocol eliminating reliance on third-party VPN daemons.

## 2. Scope and Design Philosophy

MARK is fundamentally an **orchestration, routing, and cognitive runtime** rather than a self-contained monolithic neural model. It coordinates host hardware, sensory inputs, verified tools, local runtime services (Ollama, Whisper, Piper), and authorized external providers under six core tenets:

**1. Local-First Default:** All speech recognition, conversation management, local database operations, and vision inference default to localhost runtimes without transmitting telemetry to remote servers.

**2. Principle of Least Privilege:** Dangerous capabilities (e.g., modifying system files, installing software, or accessing unrestricted shell commands) require explicit, non-bypassable graphical user confirmation.

**3. Deterministic Over Generative:** Whenever a query involves mathematical calculation, chemical properties, or calendar state, MARK routes the problem to deterministic algorithms rather than relying on generative model hallucinations.

**4. Injection Defense & Context Quarantining:** Untrusted data sources—including web search snippets, user-supplied PDFs, and external repositories—are strictly demarcated with XML-style untrusted delimiters and scanned for payload hijacking.

**5. Capability-Isolated Tooling:** External tools integrated via MCP are strictly partitioned into ReadOnly, WriteMutate, and NetworkAccess tiers, bounded by memory quotas and directory path normalization in native Rust.

**6. Resilient Zero-Trust Networking:** Inter-device communication utilizes mutual EC P-256 ECDSA authentication, transcript signatures, directional AES-256-GCM symmetric encryption, adaptive keep-alives, and automatic failover.

## 3. System Architecture

The MARK software architecture is segregated into discrete operational layers, separating UI presentation from orchestration, background execution, hardware interfacing, and network transport:

| Subsystem | Source Directory | Key Modules & Entrypoints | Status |
|---|---|---|---|
| **Core Orchestration** | src/mark/ | brain.py, intent.py, agent_loop.py, tool_registry.py | **IMPLEMENTED VERIFIED** |
| **Rust Core & MCP Bridge** | src-rust/mark_core/ | lib.rs, vault.rs, mcp/{bridge, client, security}.rs | **IMPLEMENTED TESTED** |
| **MARK Link Transport** | src-rust/mark_link/ src/mark/link/ | daemon.rs, relay.rs, identity.rs, transport.rs, service.py | **IMPLEMENTED VERIFIED** |
| **Cognition & Hybrid RAG** | src/mark/knowledge/ src/mark/cognition/ | hybrid.py, pipeline.py, indexer.py, tools.py | **IMPLEMENTED TESTED** |
| **Desktop Workstation UI** | src/mark/ui/ | main_window.py, theme_manager.py, device_hub.py | **IMPLEMENTED TESTED** |
| **Sensory & Audio** | src/mark/ | audio.py, tts.py, vision.py, vision_live.py | **IMPLEMENTED TESTED** |
| **Process Sandbox** | src/mark/evolution/ | sandbox.py, sandbox_limits.py, sandbox_harness.py | **IMPLEMENTED TESTED** |
| **Security & Vault** | src/mark/ | pki.py, provider_settings.py, logging_sanitizer.py | **IMPLEMENTED TESTED** |
| **Android Client & JNI** | mobile-android/ | MainActivity.kt, MarkLinkClient.kt, MarkLinkNative.kt | **IMPLEMENTED VERIFIED** |
| **Wear OS Companion** | mark-wear/ | MainActivity.kt, SecureBluetoothFrames.kt | **IMPLEMENTED PARTIAL** |

## 4. Request Lifecycle

A user request entering MARK follows a deterministic, synchronous-or-streaming multi-stage processing pipeline: **Stage 1: Input Ingestion & Normalization** (desktop text, Whisper voice transcription, or authenticated MARK Link packet); **Stage 2: Intent Classification & Heuristics** (lexical pattern matching and tool discovery); **Stage 3: Knowledge Retrieval & Context Assembly** (hybrid vector/lexical retrieval with untrusted content quarantine); **Stage 4: Dynamic Model Routing** (local Ollama default vs. authenticated cloud fallback); **Stage 5: Inference Execution & Streaming** (real-time token streaming with collapsible thinking tag isolation); **Stage 6: Post-Processing, Sanitization & Persistence** (regex-based secret redaction and scoped memory persistence).

## 5. Model Integration and Inference

MARK abstracts model inference behind unified provider interfaces. The primary engine is local Ollama (http://127.0.0.1:11434), serving multimodal vision-language models from the Qwen family (specifically qwen3-vl:4b and qwen3-vl:4b-instruct). Reasoning traces inside <think>...</think> tokens are dynamically extracted and rendered in collapsible viewports. External cloud providers (Gemini, Groq, OpenRouter) are invoked only upon explicit configuration, with automatic circuit breaker degradation back to local models upon network timeouts or rate limits.

## 6. Audio and Multimodal Interfaces

**Voice Capture & Synthesis:** Audio acquisition uses 16 kHz mono PCM streams with adaptive energy Voice Activity Detection (VAD). Transcription uses whisper-cli with a fast base model for wake-word detection followed by a small model

for full utterances. Neural speech synthesis is powered by local Piper ONNX voices with automatic fallback to espeak-ng.
**Visual Processing:** RTSP/RTSPS video streams from local webcams or IP cameras are captured via headless FFmpeg with strict TLS verification. Frames are constrained to an 8 MiB threshold and normalized to prevent denial-of-service before multimodal analysis.

## 7. Persistent Memory, Tools, and Authorization

**Memory Scoping:** State is partitioned across volatile session buffers, persistent encrypted agenda databases, and an inverted index knowledge base. Retrieved passages are encapsulated inside <untrusted_content> blocks to prevent indirect prompt injection.
**Deterministic Tool Registry:** Mathematical and stoichiometric calculations execute via exact algorithms (e.g., IUPAC molecular weights). Privileged system actions (modifying files, package installation) enforce non-bypassable interactive graphical authorization gates.

## 8. Native MCP Bridge & Capability Isolation

To provide high-performance, secure tool expansion without the overhead and vulnerability surface of external node daemons, MARK incorporates a native Rust Model Context Protocol implementation at src-rust/mark_core/src/mcp:

**Zero-Trust Capability Partitioning:** Tools registered through McpBridge are explicitly assigned strictly enforced capabilities: ReadOnly (read-only filesystem/inspection), WriteMutate (authorized file creation and modification within designated root paths), and NetworkAccess. Capability escalation is blocked at the Rust native boundary.

**Strict Path Normalization:** All target file paths undergo canonical path normalization in Rust. Path traversal attempts containing .., symlink escapes, or directory hopping outside authorized sandboxes trigger immediate native PermissionError exceptions.

**Memory Quotas & Anti-OOM Protection:** Payload buffers are constrained by strict size limits (default 4 MiB). Oversized JSON-RPC messages and runaway tool outputs are rejected before deserialization, safeguarding the host process from memory exhaustion attacks.

**Fuzzing-Resistant JSON-RPC Parser:** The protocol parser handles malformed envelopes, out-of-order sequence frames, and truncated payloads cleanly, returning structured JSON-RPC 2.0 error responses without panics.

## 9. Resilient Network Architecture (MARK Link)

MARK Link (implemented in Rust at src-rust/mark_link with Python orchestration at src/mark/link) delivers an independent, zero-trust transport layer between workstations, mobile nodes, and wearables:

**Multi-Tiered Connectivity Hierarchy:** Node discovery establishes communication via a deterministic priority ladder: Loopback (local IPC) > Direct Local LAN (sub-millisecond latency) > Public Reflexive (direct STUN NAT traversal) > Blind Relay (MRLY). If a direct socket drops, connection failover to the relay occurs automatically and transparently.

**Mutual Handshake & Transcript Signatures:** Handshake initiation (Type 0x01) generates ephemeral EC P-256 keypairs and 32-byte nonces. Nodes sign the canonical authentication transcript using their hardware-backed ECDSA identity key (Android Keystore). Man-in-the-middle key substitution is cryptographically impossible because transcript signatures bind ephemeral keys to authorized device identities.

**Directional Session Channels & Anti-Replay:** Following handshake completion, client and server derive independent directional AES-256-GCM symmetric keys via HKDF-SHA256. Monotonically incrementing sequence numbers and rolling window filters reject out-of-order, delayed, or replayed packets.

**Idempotency & Request Deduplication:** State-mutating requests carry unique UUIDv4 request_id headers. The daemon tracks recent request identifiers, guaranteeing that network retries or transient socket dropouts cannot cause duplicate command execution.

**Adaptive Keep-Alive & Battery Conservation:** The telemetry manager dynamically scales ping intervals: under active conversational traffic, keep-alives execute every 2-5 seconds; during idle states, intervals gracefully back off to 20-30 seconds with ±20% random jitter, immediately resetting upon new data to conserve mobile battery.

**Blind Relay Router (MRLY Protocol):** For clients behind symmetric NATs, the blind relay forwards encrypted envelopes reading only public device_id routing tags. The relay holds zero decryption keys, enforcing queue limits (64 packets per peer) and blocking open-proxy abuse.

**Immediate Revocation & Anti-Downgrade Updater:** Deauthorizing a device immediately drops all active TCP sessions. Remote software updates require cryptographically signed manifests and SHA-256 integrity checks, rejecting any lower version number (anti-downgrade enforcement).

## 10. Desktop, Mobile, and Wearable Architecture

**Desktop Workstation Client (PyQt6):** Asynchronous background worker threads (QThread), non-blocking GUI updates, rich Markdown rendering with safe URL delegation, dynamic theme switching (Dark Arc / Minimalist Tech), and modular dialogs for device management and camera feeds. *Status: IMPLEMENTED, TESTED, VERIFIED.*
**Android Client & JNI (mobile-android):** Native Android application written in Kotlin and Jetpack Compose. Interfaces with mark_link via JNI bindings (MarkLinkNative.kt), leveraging hardware Android Keystore ECDSA signing, biometric authentication, and QR-code pairing challenges. *Status: IMPLEMENTED, VERIFIED.*
**Wear OS Companion (mark-wear):** Compact smartwatch interface featuring standalone app launcher, watchface integration, and low-energy Bluetooth frame protocol. *Status: IMPLEMENTED, VERIFIED.*

## 11. Security, Privacy, and Verification

Security is implemented through defense-in-depth across execution, storage, logging, and network boundaries: **1. Kernel Namespace Sandboxing:** Dynamic code execution is isolated using Linux Bubblewrap (bwrap) with unshared user, mount, network, IPC, and PID namespaces, read-only root mounts, dropped capabilities, and volatile tmpfs filesystems; **2. Cryptographic Token Vault:** External cloud API tokens are sealed in AES-256-GCM containers derived via PBKDF2-HMAC-SHA256 (600,000 iterations); **3. Global Logging Redactor:** logging_sanitizer.py intercepts all log streams, redacting private keys, tokens, session nonces, and embedded credentials.

| Verification Category | Tool / Harness | Target Modules | Outcome |
|---|---|---|---|
| **Unit & Integration Suite** | pytest 9.1.1 | tests/ (65+ test files covering all subsystems) | **850+ PASSED<br>0 FAILED (100%)** |
| **Rust Native Unit Tests** | cargo test | mark_core (14 tests) & mark_link (58 tests) | **72 PASSED<br>0 FAILED (100%)** |
| **Static Security Analysis** | Bandit 1.9.4 | src/ (Python codebase) | **0 HIGH / 0 MEDIUM<br>(Clean Audit)** |
| **Credential & Secret Scan** | detect-secrets / AST | Entire repository tree (zero leaks) | **0 SECRETS<br>0 HOST IPS** |
| **Cognitive Benchmarking** | MARKBENCH (v0.1-v0.3) | tests/markbench/ cognitive regression suite | **EVALUATED<br>VERIFIED** |

## 12. Deployment, Operations, and Conclusion

**Operations:** MARK runs natively on Linux workstations with Python 3.10-3.14 and Rust. Settings reside in ~/.config/mark/config.json (mode 0600). Systemd user units (mark-desktop.sh, mark-mobile.service) manage background execution.

**Conclusion:** The public distribution of MARK proves that advanced local artificial intelligence, multimodal comprehension, extensible native MCP tool execution, and secure mobile federation can be achieved without compromising user privacy or host integrity.

### References

[1] Rescorla, E. 'The Transport Layer Security (TLS) Protocol Version 1.3', RFC 8446, August 2018.

[2] Dworkin, M. 'Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)', NIST SP 800-38D, 2007.

[3] Model Context Protocol (MCP) Specification, Anthropic / Open Source Standard, 2024.

[4] Project Atomic / Red Hat. 'Bubblewrap: Unprivileged Sandboxing Tool', https://github.com/containers/bubblewrap.

[5] PyO3 Project. 'Rust bindings for the Python interpreter', https://github.com/PyO3/pyo3.

[6] Radford, A. et al. 'Robust Speech Recognition via Large-Scale Weak Supervision' (Whisper), OpenAI, 2022.

[7] Qwen Team. 'Qwen2.5-VL: Technical Report on Vision-Language Understanding', Alibaba Cloud, 2025.